

Secțiunea soft utilitar

TurboGrafx-16

Sistem de jocuri video

Elev: Buciuman Tudor, clasa a XII-a

Prof. îndrumător: Alin-Doru Maidan

Colegiul Național „Vasile Lucaciu” Baia Mare

Anul școlar 2025-2026





TUDORTALES GAMES

TURBOGRAFX 16

The system starts without explanation.

A simple interface. A list of names. No instructions, no context—just something that feels familiar enough to trust. You select a program, and it begins immediately. Movement is precise. Rules are clear. Every action has a response, every failure a reason. It doesn't take long before the structure starts to make sense.

In here, things behave the way they're supposed to.

Progress is visible. Effort is measurable. If something doesn't work, you try again. And again. It's consistent. Predictable. Fair, in a way that doesn't need to be explained.

Outside, things are less certain.

There are no defined paths, no clear objectives, no indication that you're doing anything correctly. The system doesn't respond the same way. Actions don't always lead to results. Time passes differently. It's harder to tell when something is finished—or if it ever was.

But the patterns stay with you.

What you learn here doesn't disappear. It lingers. It shapes how you move, how you decide, how you interpret what's in front of you. The structure becomes familiar enough that you begin to look for it elsewhere.

And sometimes, it almost feels like it's there.

TurboGrafx-16 presents a contained experience. A controlled space where everything operates within a defined logic. But as that logic begins to extend beyond its boundaries, the distinction between system and reality becomes nonexistent.

There is no explicit transition. No clear moment where one ends and the other begins.

Only the feeling that something is carrying over.

There's only one way to win, and you'll know what you have to do. What choices to make. What to say. What to think. Only at the end does it become clear that this was not merely happening. That it was being carried. That it was passing through a voice, worn thin by the telling. I, too, have been touched by the devilish one. ...oh, but when I saw the face of God, I was changed... Took the entire arm off!

Unleash the Higher Energy.

Capitole

1. Recontextualizarea sistemelor reactive non-deterministe în arhitecturi retro inspirate de sistemele TurboGrafX-16/PC Engine	3
2. BITROOT: Arhitectură modulară într-un joc retro	4
3. Arhitectură și structură	5
4. Tehnici de programare.....	6
5. Fizică și sisteme modulare.....	7
6. Inteligența sistemelor inamice.....	10
7. Poveste	13
8. Direcție artistică.....	14
9. Cerințe de sistem	15
10. Tehnologii.....	16
11. Concluzie.....	17
12. Bibliografie.....	18
13. Screenshot-uri.....	19

*An origin buried in darkness, pulling the surface down to feed its growth. It is the anchor we cannot see, the first cause of every branching path, holding fast beneath the floorboards. We claim the light of the crown, but the foundation is a tangled net in the deep, drinking the secrets of the soil until the whole world stems from a single, forgotten seed. You can run from the dark or try to break free, but in the end, every single route leads right back to the seed planted in the deep. We chose the branches, but the blade holds the ultimate truth, that every path we take is watered by the same forgotten **roots**.*

1. Recontextualizarea sistemelor reactive non-deterministe în arhitecturi retro inspirate de sistemele TurboGrafx-16/PC Engine

BITROOT este un minijoc arcade conceput pentru a recrea atmosfera specifică perioadei de tranziție dintre generațiile de console pe 8 și 16 biți, inspirându-se în mod direct din identitatea vizuală și tehnică a platformei TurboGrafx-16, cunoscută în Japonia sub numele de PC Engine. Obiectivul principal al proiectului a fost reproducerea fidelă a farmecului tehnologic specific erei retro, prin limitări vizuale intenționale, efecte grafice inspirate de televizoarele de tip CRT și o prezentare artistică apropiată de jocurile arcade ale sfârșitului anilor '80 și începutului anilor '90. Deși jocul a fost realizat utilizând motorul modern Unity, întregul proces de dezvoltare a urmărit simularea constrângerilor hardware caracteristice platformelor istorice. Consola a fost lansată în SUA în a doua jumătate din 1989.

Pentru a păstra autenticitatea esteticii retro, am utilizat o rezoluție fixă de 256×224 pixeli și un aspect de 4:3, replicând standardele video utilizate de consolele clasice ale perioadei. Această abordare contribuie la obținerea unei imagini apropiate de experiența oferită de televizoarele CRT, unde claritatea redusă, scanline-urile și imperfecțiunile analogice reprezentau o componentă definitorie a identității vizuale. În plus, interfața și sprite-urile au fost create ținând cont de limitările grafice ale erei, evitând elementele moderne precum anti-aliasing-ul sau rezoluțiile foarte mari sau DLSS șamd.

Un element important al designului îl reprezintă paleta de culori. Consola TurboGrafx-16 era recunoscută pentru capacitatea sa de a afișa culori vibrante și saturate, apropiate de sistemele arcade dedicate, având posibilitatea de a reda simultan până la 482 de culori pe ecran. Din acest motiv, BITROOT utilizează o direcție artistică bazată pe contraste puternice, tonuri neon și combinații cromatice inspirate de jocurile japoneze arcade ale perioadei. Această alegere contribuie atât la autenticitatea estetică, cât și la atmosfera digitală instabilă și experimentală pe care jocul încearcă să o transmită.

Din punct de vedere tehnic, consola TurboGrafx-16 reprezenta un caz aparte în istoria hardware-ului retro. Deși era comercializată drept o platformă pe 16 biți datorită subsystemului său grafic avansat, procesorul central rămânea unul pe 8 biți. Această combinație genera un contrast interesant între puterea vizuală ridicată și limitările de procesare ale sistemului. Comparativ cu NES, platforma oferea performanțe grafice cu mult superioare, însă în anumite situații rămânea inferioară sistemelor complet pe 16 biți precum SNES. Cu toate acestea, arhitectura bazată pe două procesoare grafice de 16 biți îi permitea să afișeze sprite-uri detaliate, culori numeroase și efecte vizuale impresionante pentru perioada respectivă, aspecte care au influențat în mod direct stilul artistic și tehnic al proiectului BITROOT.

2. BITROOT: Arhitectură modulară într-un joc retro

BITROOT utilizează o arhitectură modulară concepută pentru a separa toate sistemele importante ale jocului în componente independente și reutilizabile. Această abordare permite dezvoltarea simultană a mai multor experiențe interactive complet diferite, păstrând însă aceeași infrastructură tehnică de bază. Proiectul nu a fost gândit exclusiv ca un simplu minijoc retro, ci ca un ecosistem digital fragmentat, în interiorul căruia pot exista lumi, stiluri vizuale și mecanici radical diferite, toate conectate prin conceptul narativ de **ROOTS**.



Din punct de vedere conceptual, **ROOTS** reprezintă infrastructura digitală care leagă toate experiențele interactive ale proiectului. Fiecare “rădăcină” funcționează ca o ramificație procedurală a aceluiași sistem corupt, permițând existența unor jocuri cu identități vizuale și mecanice complet distincte. Astfel, **BITROOT** reproduce atmosfera arcade a platformelor TurboGrafx-16 și PC Engine, în timp ce alte secțiuni ale ecosistemului pot adopta structuri moderne inspirate de jocuri open-world din ziua de azi, apropiate conceptual de experiențe precum *Cyberpunk 2077*. Această modularitate narativă și tehnică permite coexistența mai multor paradigme vizuale fără a afecta coerența generală a proiectului.

Un element central al arhitecturii îl reprezintă sistemul de gestionare al camerelor și al ecranelor de joc, inspirat direct de structura utilizată în *The Legend of Zelda* pentru NES. În locul unei camere complet libere, lumea este împărțită în segmente fixe, fiecare reprezentând o „cameră” independentă conectată procedural cu celelalte. Atunci când jucătorul atinge marginea unei zone, camera realizează o tranziție controlată către următorul segment al hărții, păstrând astfel senzația specifică jocurilor clasice.

Această metodă de organizare oferă mai multe avantaje tehnice și estetice. În primul rând, reduce semnificativ complexitatea procesării simultane a obiectelor și permite încărcarea controlată a entităților active. În al doilea rând, contribuie la autenticitatea experienței retro, deoarece limitează câmpul vizual al jucătorului exact precum sistemele hardware istorice. În plus, fiecare cameră poate funcționa ca un modul aproape independent, având propriile reguli, inamici, efecte vizuale și atmosferă, fără a influența direct celelalte segmente ale jocului. La nivel software, această arhitectură este implementată prin componente independente bazate pe sistemul MonoBehaviour din Unity. Fiecare subsistem, gestionarea camerelor, efectele CRT, logica inamicilor, interfața sau tranzițiile vizuale, este separat în clase distincte pentru a reduce dependențele și pentru a facilita extinderea ulterioară a proiectului. Astfel, **BITROOT** reușește să combine limitările estetice ale erei retro cu flexibilitatea arhitecturilor software moderne, motorul Unity, păstrând atât autenticitatea vizuală, cât și dimensiunea tehnică necesară dezvoltării unor experiențe interactive multiple.

3. Arhitectură și structură

Cu toate că versiunea originală de TurboGrafx-16, cel puțin cea lansată în SUA, avea procesorul de 8 biți, este destul de evident că nu am cum să implementez un joc de stil 8 bit în 2023 în Unity. Problema evidentă este faptul că Unity folosește aproape peste tot valori de tip float, care nu pot fi reprezentate decât sub formă de fracții simple. Așadar, pot spune că practic am trișat folosind Unity pentru un joc 2D, dar totodată este foarte ușor de creat, programat, desenat, importat muzică și evident compilat (fiind cel mai rapid engine la această scară).

Arhitectura proiectului BITROOT a fost concepută într-o manieră modulară, având ca obiectiv principal separarea logicii de gameplay, a componentelor vizuale și a sistemelor auxiliare. Proiectul utilizează structura bazată pe componente specifică motorului Unity, unde fiecare entitate importantă din joc este controlată prin scripturi independente de tip MonoBehaviour. Această abordare permite organizarea clară a funcționalităților, simplifică procesul de mentenanță și facilitează extinderea ulterioară a proiectului fără modificări majore asupra sistemelor deja existente.

În ceea ce privește organizarea scenelor, BITROOT utilizează o separare clară între meniuri, gameplay și secvențele speciale ale minijocului. Această metodă contribuie la încărcarea eficientă a resurselor și permite controlul precis asupra elementelor active în timpul execuției. Totodată, arhitectura proiectului urmărește simularea limitărilor tehnice specifice erei TurboGrafx-16, atât la nivel vizual, cât și la nivel de procesare, păstrând însă avantajele oferite de un motor modern de dezvoltare (Unity).

Structura internă a jocului este împărțită în mai multe subsisteme principale, printre care sistemul de randare, logica de gameplay, controlul inamicilor, gestionarea interfeței și efectele vizuale retro. Comunicarea dintre aceste componente este realizată prin evenimente, verificări de stare și actualizări sincronizate în cadrul ciclului principal de execuție al motorului Unity. Sistemele reactive precum coliziunile, knockback-ul sau efectele CRT sunt implementate independent pentru a reduce dependențele dintre clase și pentru a păstra o structură flexibilă și ușor de optimizat.

Totodată, alte funcționalități pe care le-am implementat sunt: sistemul inamicilor modular inspirat de The Legend of Zelda, în care inamicii au 4 stări posibile: pașnic, paznic, agresiv și mod de luptă, iar acești inamici se împart în alte categorii, precum arcaș, vânător și vrăjitor.



Tranziția între camere este inspirată evident de Zelda, iar la părăsirea camerei toți inamicii în viață își vor recupera viața și reveni la locurile lor de plecare. În starea agresivă se vor deplasa în întreaga cameră căutându-te, iar paznicii vor putea redirecționa ceilalți inamici pe tine. Iar dacă ai crezut că e prea dificil, am implementat și o modalitate de a te putea vindeca: aleatoriu vor lăsa în urma lor inimi, fiecare inamic cu propria sa probabilitate de a le lăsa.

4. Tehnici de programare

Una dintre cele mai bune idei ale mele a fost codul pentru gestionarea traversării între camere printr-o Coroutină care are ca rol tranziția camerei în direcția indicată, dezactivarea momentană a mișcării jucătorului, mișcându-se de unul singur, iar apoi camera parăsită este actualizată pentru inamici.

```
1. private void OnTriggerEnter2D(Collider2D other)
2. {
3.     if (triggered || inUse) return;
4.     if (!other.CompareTag("Player")) return;
5.     //detectează când jucătorul atinge colțul unei uși
6.     RoomCamera cam = Camera.main.GetComponent<RoomCamera>();
7.     PlayerMovement player = other.GetComponent<PlayerMovement>();
8.     if (cam.IsMoving())
9.         return;
10.    triggered = true;
11.    StartCoroutine(Transition(cam, player));
12. }
13.
14. IEnumerator Transition(RoomCamera cam, PlayerMovement player)
15. {
16.     inUse = true;
17.     // Mișcarea camerei
18.     cam.MoveToRoom(direction, speed);
19.     player.MoveToTheOtherRoom(direction);
20.     // Aștept să termine camera de traversat în următoarea cameră
21.     while (cam.IsMoving())
22.         yield return null;
23.     yield return new WaitForSeconds(controlDelay);
24.     player.enabled = true;
25.     HealItem[] t = FindObjectsOfType<HealItem>();
26.     foreach (HealItem a in t)
27.     {
28.         a.DestroyOnExit();
29.     }
30.     triggered = false;
31.     player.Reenable();
32.     inUse = false;
33. }
34.
```


5. Fizică și sisteme modulare

```
1. void Update()
2. {
3.     if (canMove)
4.     {
5.         float moveX = UTInput.GetAxisRaw("Horizontal");
6.         float moveY = UTInput.GetAxisRaw("Vertical");
7.         if (Moved())
8.         {
9.             anim.SetFloat("speed", 1);
10.            anim.SetBool("isMoving", value: true);
11.            moveDir = new Vector3(UTInput.GetAxisRaw("Horizontal"),
UTInput.GetAxisRaw("Vertical"));
12.            if (moveDir != Vector3.zero)
13.            {
14.                if (new List<Vector2> { Vector2.up, Vector2.left, Vector2.down,
Vector2.right }.Contains(moveDir))
15.                {
16.                    faceDir = moveDir;
17.                }
18.                else if (0f - moveDir.x == faceDir.x || 0f - moveDir.y == faceDir.y)
19.                {
20.                    faceDir = new Vector3(0f, moveDir.y);
21.                }
22.                ChangeDirection(faceDir);
23.            }
24.        }
25.        else
26.        {
27.            anim.SetBool("isMoving", value: false);
28.        }
29.        movement = Vector2.Lerp(movement, new Vector2(moveX, moveY), 0.2f);
30.    }
31. }
32. public float GetAxisRaw(string s)
33. {
34.     if (s == "H")
35.     {
36.         return Input.GetAxisRaw("Horizontal");
37.     }
38.     else if (s == "V")
39.     {
40.         return Input.GetAxisRaw("Vertical");
41.     }
42.     return 0;
43. }
44. public void ChangeDirection(Vector2 dir)
45. {
46.     anim.SetFloat("dirX", dir[0]);
47.     anim.SetFloat("dirY", dir[1]);
48. }
49. private bool Moved()
50. {
51.     if (UTInput.GetAxisRaw("Horizontal") == 0f)
52.     {
53.         return UTInput.GetAxisRaw("Vertical") != 0f;
```

```

54.     }
55.     return true;
56. }
57. public Vector2 GetDirection()
58. {
59.     return new Vector2(anim.GetFloat("dirX"), anim.GetFloat("dirY"));
60. }
61. public void MoveToTheOtherRoom(Vector2 v)
62. {
63.     canMove = false;
64.     movement = Vector2.zero;
65.     currentRoom += (int)v.x;
66.     currentRoom += (int)v.y*roomsWidth;
67.     EnemyMovement[] t = FindObjectsOfType<EnemyMovement>();
68.     foreach(EnemyMovement a in t)
69.     {
70.         a.OnPlayerExitRoom();
71.     }
72.     foreach (EnemyMovement a in t)
73.     {
74.         if (a.GetRoom() == GetCurrentRoom())
75.             a.ResetEnemy();
76.     }
77.     StartCoroutine(MoveAFewPixels(v, 0.8f));
78. }
79. public int GetCurrentRoom()
80. {
81.     return currentRoom;
82. }
83. public IEnumerator MoveAFewPixels(Vector2 v, float duration)
84. {
85.     ChangeDirection(v);
86.
87.     anim.SetFloat("speed", 1);
88.     anim.SetBool("isMoving", true);
89.     canMove = false;
90.
91.     float timer = 0f;
92.     float speed = 4f;
93.
94.     while (timer < duration)
95.     {
96.         //transform.position += new Vector3(v.x, v.y, 0) * speed * Time.deltaTime;
97.         movement = Vector2.Lerp(movement, v, 0.2f);
98.         timer += Time.deltaTime;
99.         yield return null;
100.    }
101.    movement = Vector2.zero;
102.    anim.SetBool("isMoving", false);
103. }
104. public void Reenable()
105. {
106.     canMove = true;
107. }
108. public void MovementToZero()
109. {
110.     movement = Vector2.zero;
111. }
112. public Vector2 GetFaceDirection()
113. {

```

```

114.     return faceDir;
115. }
116. public void ApplyKnockback(Vector2 sourcePosition)
117. {
118.     if (!gameObject.activeInHierarchy || isKnockedBack)
119.         return;
120.
121.     StartCoroutine(KnockbackRoutine(sourcePosition));
122. }
123. IEnumerator KnockbackRoutine(Vector2 sourcePosition)
124. {
125.     isKnockedBack = true;
126.     canMove = false;
127.
128.     anim.SetBool("isMoving", false);
129.
130.     Vector2 dir = ((Vector2)transform.position - sourcePosition).normalized;
131.
132.     float timer = 0f;
133.
134.     while (timer < knockbackDuration)
135.     {
136.         rb.velocity = dir * knockbackForce;
137.
138.         timer += Time.deltaTime;
139.         yield return null;
140.     }
141.
142.     rb.velocity = Vector2.zero;
143.     movement = Vector2.zero;
144.
145.     canMove = true;
146.     isKnockedBack = false;
147. }
148. void FixedUpdate()
149. {
150.     if (!isKnockedBack)
151.     {
152.         rb.velocity = movement * moveSpeed;
153.     }
154.
155.     float pixelsPerUnit = 16f;
156.
157.     Vector3 pos = transform.position;
158.     pos.x = Mathf.Round(pos.x * pixelsPerUnit) / pixelsPerUnit;
159.     pos.y = Mathf.Round(pos.y * pixelsPerUnit) / pixelsPerUnit;
160.     transform.position = pos;
161. }

```

Aceste funcții sunt responsabile de fizica caracterului, fiind de asemenea de tip 8 bit. Viteza este calculată dacă jucătorul interacționează sau în momentul în care este atacat sau când trece prin camere. Animațiile sunt realizate în totalitate prin funcțiile de animație din Unity.

6. Inteligența sistemelor inamice

Sistemul inamic se bazează pe un comportament determinist bazat pe stări, implementat prin intermediul framework-ului MonoBehaviour al Unity. Fiecare inamic operează folosind funcții de joc predefinite, inclusiv mișcare, detectare, execuție a atacurilor și mod de knockback reactiv. Această abordare asigură un comportament de joc previzibil, menținând în același timp o complexitate computațională redusă și o performanță stabilă în timpul rulării.

```
1. void Start()
2. {
3.     player = GameObject.FindGameObjectWithTag("Player").transform;
4.     startPosition = transform.position;
5.     if (animState == 1)
6.     {
7.         MadEnemy();
8.     }
9. }
10.
11. void Update()
12. {
13.     if (!playerInRoom || DefaultState==State.Idle) return;
14.     float distToPlayer = Vector2.Distance(transform.position, player.position);
15.
16.     if (isKnockedBack && DefaultState!=State.Idle)
17.     {
18.         transform.position += (Vector3)(knockbackVelocity * Time.deltaTime);
19.
20.         knockbackTimer -= Time.deltaTime;
21.         if (knockbackTimer <= 0)
22.         {
23.             isKnockedBack = false;
24.         }
25.
26.         return;
27.     }
28.     if (playerInRoom && distToPlayer < detectionRange && (DefaultState==State.Patrol ||
DefaultState == State.Chase))
29.     {
30.         currentState = State.Chase;
31.     }
32.
33.     switch (currentState)
34.     {
35.         case State.Idle:
36.             break;
37.
38.         case State.Patrol:
39.             Patrol();
40.             break;
41.
42.         case State.Chase:
43.             Chase();
44.             break;
45.
46.         case State.Return:
47.             ReturnToStart();
48.             break;
```

```

49.     }
50. }
51.
52. void Patrol()
53. {
54.     if (patrolPoints.Length == 0) return;
55.     Transform target = patrolPoints[patrolIndex];
56.
57.     MoveTowards(target.position);
58.
59.     if (Vector2.Distance(transform.position, target.position) < 0.2f)
60.     {
61.         patrolIndex = (patrolIndex + 1) % patrolPoints.Length;
62.     }
63. }
64. public void OnPlayerExitRoom()
65. {
66.     playerInRoom = false;
67.     currentState = State.Return;
68. }
69. public void ResetEnemy()
70. {
71.     transform.position = startPosition;
72.     playerInRoom = true;
73.     currentState = DefaultState;
74.     GetComponent<Enemy>().ResetHealth();
75. }
76. public int GetRoom()
77. {
78.     return room;
79. }
80. public void MadEnemy()
81. {
82.     this.GetComponent<Animator>().Play("enemy2");
83. }
84. void ReturnToStart()
85. {
86.     MoveTowards(startPosition);
87.
88.     if (Vector2.Distance(transform.position, startPosition) < 0.1f)
89.     {
90.         transform.position = startPosition;
91.         currentState = State.Idle; //sau o Patrulă, la alegerea mea
92.     }
93. }
94. void Chase()
95. {
96.     float dist = Vector2.Distance(transform.position, player.position);
97.
98.     if (dist > detectionRange * 1.5f)
99.     {
100.         currentState = State.Patrol;
101.         return;
102.     }
103.
104.     if (dist > stopDistance)
105.     {
106.         MoveTowards(player.position);
107.     }
108.     else

```

```

109.     {
110.         return;
111.     }
112. }
113. public void ApplyKnockback(Vector2 direction, float force, float duration)
114. {
115.     isKnockedBack = true;
116.     knockbackTimer = duration;
117.     knockbackVelocity = direction * force;
118. }
119. void MoveTowards(Vector2 target)
120. {
121.     Vector2 dir = (target - (Vector2)transform.position).normalized;
122.     //dir = Vector2.Lerp(Vector2.zero, dir, 0.8f);
123.     transform.position += (Vector3)(dir * moveSpeed * Time.deltaTime);
124. }
125. }
126. void OnCollisionStay2D(Collision2D collision)
127. {
128.     if (collision.gameObject.CompareTag("Player") && DefaultState != State.Idle)
129.     {
130.         if (Time.time > lastDamageTime + damageCooldown)
131.         {
132.             PlayerHealth hp = collision.gameObject.GetComponent<PlayerHealth>();
133.
134.             if (hp != null)
135.             {
136.                 hp.TakeDamage(damage);
137.                 lastDamageTime = Time.time;
138.             }
139.         }
140.     }
141. }
142. private void OnCollisionEnter2D(Collision2D collision)
143. {
144.     if (collision.gameObject.CompareTag("Player") && DefaultState != State.Idle)
145.     {
146.         PlayerMovement p = collision.gameObject.GetComponent<PlayerMovement>();
147.
148.         if (p != null)
149.         {
150.             p.ApplyKnockback(transform.position);
151.         }
152.     }
153. }
154.

```

Acesta este unul dintre componentele de bază al unui inamic care va putea apărea într-una din camerele din BITROOT. Pe măsură ce jucătorul evoluează și inamicii vor fi pe măsură. Sistemul de progresie are în total 5 nivele, care afectează atacul, viața jucătorului și viteza de atac.



7. Poveste

Povestea începe în interiorul unei camere întunecate, ca un buncăr. Tot ce are la dispoziție în prima fază jucătorul este sabia. Pe măsură ce se duce tot mai adânc, va vedea că jocul a fost de la bun început un RPG. Vei vâna. Vei ataca. Vei câștiga. Dar cu ce preț?

Cum am promis (să dezvălui povestea la InfoEducație), firul narativ al jocului se împarte în 2 mari acte: **BITROOT**, care reprezintă fantezia, totodată înseamnă și evadarea din viața cotidiană într-o lume complet virtuală, unde greșelile pot fi mereu evitate, și **Turbograful**.

Pe scurt, BITROOT este un joc pe care personajul fictiv îl joacă pe consola video retro din 1989: TurboGrafx-16, (existând în Turbograf o consolă de tip 3d, creată de mine în Blender, pe care personajul o folosește să joace mai multe jocuri, jucătorul: adică noi avem acces doar la BITROOT). Universul fictiv pe care mi l-am imaginat poartă numele de Turbograf, acțiunea având loc pe perioada a mai multor luni, la sfârșit de secol XXI. Tehnologia este foarte avansată, purtând omagiu celui mai bun RPG din istorie: Cyberpunk 2077, dar cu evidenta observație că acțiunea începe abia după ce a fost completat BITROOT, iar abia după jucătorul va afla că mai puțin de 1% din adevăratul gameplay a fost BITROOT.

Turbograful este foarte complex pentru arhitectura telefoanelor, așadar am creat 2 versiuni diferite: cea care conține doar BITROOT, care este singura versiune accesibilă pe android, iar Turbograful, care conține ambele acte. Pentru a declanșa al doilea act trebuie bătut inamicul final din BITROOT, iar apoi jucătorul se va distanța treptat de CRT și de pe geamul camerei se va putea vedea un oraș cu adevărat plasat la sfârșitul secolului XXI. Lângă el se va afla iubita protagonistului, privind gânditoare peste civilizația industrială (urbanizată) și asupra corporațiilor multinaționale care oglindesc situația din Cyberpunk 2077. Privind la protagonist, aceasta îi dezvăluie de o boală foarte gravă pe care a ascuns-o de el în tot acest timp. Îi cere scuze că nu i-a spus mai devreme și îi cere ca după sfârșitul ei să prețuiască mai mult timpul real pe care îl are, deoarece nu se știe niciodată cât timp mai are. La finalul scenei, amândoi privesc cerul de pe fereastră. Evident că în scena următoare, plasată la câteva luni mai târziu protagonistul privește trist la poza ei, fiind clar că s-a lăsat de jocurile video, deci jucătorul nu se va mai putea juca niciodată BITROOT.

Concluzia jocului este una destul de evidentă: trăiește clipa. Trebuie să apreciem mai mult viața și să nu ne mai izolăm în acest mediu digital. Totodată, BITROOT are și el o concluzie: violența nu este soluția. Această idee este nepotrivită pentru un RPG unde nici nu ai de ales: violența este obligatorie pentru a progresa. Vioența este mai mult decât necesară pentru a câștiga, dar asta nu înseamnă că finalul este unul fericit. Ideea de rădăcină este că necesită foarte multă putere pentru a fi ruptă, putere care o poți obține doar luptând. Tot violența este de asemenea cea care pune capăt jocului: caracterul pe care l-am controlat în BITROOT a devenit atât de puternic încât își întoarce sabia către noi.

8. Direcție artistică

Direcția artistică a proiectului BITROOT urmărește reconstruirea identității vizuale și emoționale specifice perioadei consolelor arcade japoneze de la sfârșitul anilor '80 și începutul anilor '90. Stilul vizual este inspirat direct de platforma **TurboGrafx-16**, cunoscută în Japonia sub numele de PC Engine, sistem recunoscut pentru culorile sale vibrante, sprite-urile detaliate și atmosfera apropiată de jocurile arcade ale perioadei. Totuși, proiectul nu încearcă doar să reproducă limitările tehnice ale hardware-ului retro, ci să transforme aceste limitări într-un limbaj artistic și narativ propriu, în stil 8 bit și 64 bit.

Din punct de vedere tematic, BITROOT explorează ideea degradării progresive a memoriei digitale și a trecerii ireversibile a timpului. Primele secvențe ale jocului sunt construite în jurul unei atmosfere aparent familiare și aventuroase, inspirate de jocurile clasice de explorare, însă pe măsură ce experiența avansează, spațiile interactive devin tot mai instabile, mai fragmentate și mai lipsite de claritate. Tranziția dintre aventura retro și sentimentul de nostalgie melancolică este realizată gradual, fără schimbări bruște, tocmai pentru a crea impresia unei lumi care se degradează lent sub propria existență digitală.

Această transformare vizuală este susținută prin modificări subtile ale paletelor cromatice, iluminării și efectelor de post-procesare. Zonele inițiale utilizează culori saturate și contraste puternice, evocând perioada de glorie a jocurilor arcade, în timp ce secțiunile ulterioare introduc tonuri mai reci, distorsiuni CRT, instabilitate vizuală și artefacte electronice inspirate de televizoarele analogice defecte. Astfel, degradarea tehnică devine o componentă narativă activă, nu doar un efect estetic.

Un rol important în definirea atmosferei îl are componenta audio. Prima etapă a jocului utilizează ca fundal muzical piesa **Idiotique**, a trupei Radiohead, alegere destul de evidentă. Ritmul mecanic și structura repetitivă a piesei contribuie la senzația de instabilitate și anticipează degradarea progresivă a universului digital prezentat în joc. În acest context, muzica funcționează ca parte integrată a construcției emoționale și narrative a experienței.

Motivul de ROOTS este de asemenea o componentă importantă în Turbograf, fiind cheia de înțelegere a întregului concept.



9. Cerințe de sistem

Pentru a putea rula TurboGrafx-16, Unity are anumite cerințe de sistem precum DirectX 10, precum și de hardware. Acestea sunt cerințele:

1) Hardware:

- **CPU:** Intel® Core™ i3 gen 10+
- **GPU** sau integrată
- **RAM:** 4GB DDR4
- **HDD:** 140-150 mb

2) Software:

- **OS:** Windows 10/11 sistem de 64 de biți
- **DirectX** 10+

Sistemul meu prezintă:

1) Hardware:

- **CPU:** Intel® Core™ i5-12400f 2.5GHz Turbo: 4.5GHz
- **GPU:** AMD RX 6600
- **RAM:** 16GB DDR4 3600MHz
- **HDD:** 2TB 5400rpm

2) Software:

- **OS:** Windows 11, versiunea oficială din aprilie 2026
- **DirectX** 10+

Totodată, TurboGrafx-16 are o versiune specială pentru Android. Eu l-am rulat cu succes pe android-ul meu de versiune Android 16. Aceasta are cerințe minime, precum:

- **CPU:** ARMv7 sau ARM64
- **GPU:** AMD RX 6600
- **RAM:** 2GB
- **Versiune:** Android 6.0 , până la Android 16 (deci sdk 36)
- **HDD:** 150 (fișierul deși are doar 50mb, necesită 150mb)

10. Tehnologii

Dezvoltarea jocului s-a bazat pe un cadru tehnologic modern, care integrează un motor de joc profesional, chiar cel mai folosit motor de jocuri modern al secolului XXI, **Unity**, folosindu-mă de un limbaj de programare robust și un set extins de biblioteci standard, adică de **C#**. Această combinație a permis atât o implementare eficientă (nu cred că există alt limbaj compatibil în Unity), cât și flexibilitatea pentru explorarea unor mecanici de joc neconvenționale. Pentru Turbograf am folosit Unity 2D versiunea 2023.1.20f. Arhitectura Unity este bazată pe un model orientat pe componente, în care funcționalitatea este încapsulată în scripturi reutilizabile atașate obiectelor din scenă, fără de care nu cred că aş fi avut vreo şansă.

Limbajul principal de programare utilizat în cadrul proiectului este C#, care constituie baza scripting-ului în Unity. C# este un limbaj orientat pe obiect, cu tipizare statică, ce oferă un echilibru între performanță, siguranță și expresivitate. Caracteristicile sale orientate pe obiect, precum încapsularea, moștenirea și polimorfismul, ce au fost esențiale pentru modelarea structurii ierarhice a entităților din joc, în special pentru reprezentarea diferitelor tipuri de piese de șah și a comportamentelor acestora. De asemenea, sintaxa bine definită a contribuit la creșterea mentenanței codului și la reducerea erorilor de execuție.

Complementar limbajului propriu-zis, proiectul utilizează bibliotecile standard furnizate de Microsoft. Aceste biblioteci oferă o gamă largă de funcționalități predefinite, inclusiv structuri de date, operații de intrare/ieșire și utilitare matematice. În contextul jocului BITROOT, colecțiile precum listele și dicționarele au avut un rol esențial în gestionarea stărilor dinamice ale jocului, grafica 2d+3d și **implementarea variațiilor de reguli**. Utilizarea acestor instrumente standardizate a contribuit la creșterea eficienței dezvoltării și la îmbunătățirea fiabilității jocului, reducând necesitatea implementării unor funcționalități de bază de la zero.

În procesul de dezvoltare a fost utilizat sistemul de versionare Git, prin intermediul platformei GitHub, pentru gestionarea eficientă a codului sursă și urmărirea modificărilor. Toate proiectele mele sunt încărcate pe contul meu de Github, unde sunt publice, iar apoi executabilele sunt încărcate pe Itch.io.

Totodată, am lucrat în Visual Studio 2022, datorită integrării incredibile cu Unity.

Alte tehnologii folosite includ GIMP, aplicație de editare pentru imagini, cu care am realizat texturile, iar anumite texturi le-am descărcat tot de pe Itch.io.

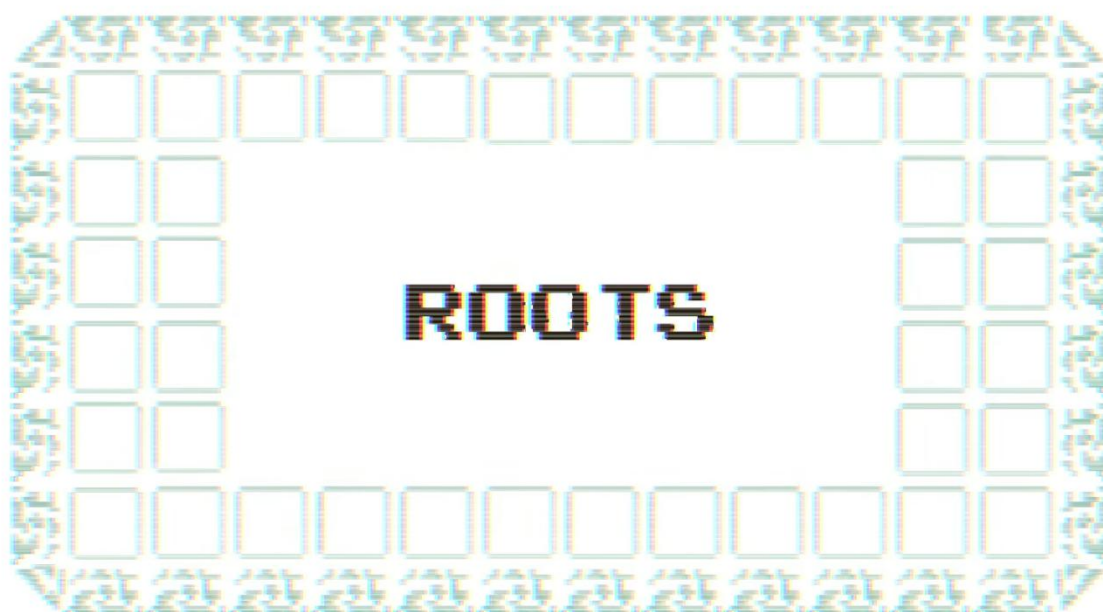
11. Concluzie

În concluzie, proiectul TurboGrafx-16 / BITROOT reprezintă o explorare a intersecției dintre tehnologie, estetică retro și narațiune interactivă, unde designul jocului devine un instrument de exprimare artistică. Prin îmbinarea elementelor de gameplay cu o direcție artistică puternic ancorată în nostalgie și degradare temporală, proiectul reușește să transmită o experiență simbolică, în care lumea jocului reflectă idei mai profunde despre memorie, schimbare și pierdere.

Din punct de vedere tehnic, implementarea sistemelor de gameplay și a structurii open-world demonstrează modul în care deciziile de arhitectură software influențează direct experiența jucătorului. Totodată, utilizarea unor tehnologii moderne în contrast cu o estetică inspirată din generații hardware mai vechi subliniază intenția de a crea un dialog între trecut și prezent.

Astfel, TurboGrafx / BITROOT nu este doar un proiect de joc, ci și o formă de experiment artistic și tehnologic, care își propune să reinterpreteze limbajul jocurilor video ca mediu narativ și expresiv.

Și astfel, în concluzie, tot ce am observat indică un singur adevăr inevitabil: modelul nu a fost niciodată aleatoriu. Fracturile s-au aliniat prea perfect, ecourile s-au repetat prea precis, rezultatele s-au întors ca și cum ar fi fost trase de o gravitație nevăzută. Sub fiecare alegere se afla o altă versiune a acesteia, îngropată ca o fundație veche pe care nimeni nu s-a oboșit să o cartografieze. La început, am crezut că acestea sunt coincidențe - suprapuneri slabe, scurgeri inofensive. Iar apoi sunetul s-a auzit venind din jos. Pereții au început să-și amintească. Și umbrele au început să mă urmeze. Am urmat ultimul fir în jos, așteptând un răspuns. Ceea ce am găsit a fost o formă întunecată, crescută din rădăcinile fiecărei căi pe care nu am urmat-o. Deja nu mai eram singur..



12. Bibliografie

Algoritmică

<https://www.geeksforgeeks.org/dsa/design-a-chess-game/>

<https://www.sciencedirect.com/science/article/pii/S0950584923001179>

<https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/>

<https://www.geeksforgeeks.org/artificial-intelligence/mini-max-algorithm-in-artificial-intelligence/>

<https://towardsdatascience.com/building-a-chess-ai-that-learns-from-experience-5cff953b6784/>

<https://en.wikipedia.org/wiki/TurboGrafx-16>

<https://archive.org/details/turbografx-adverts>

Documentație

[https://en.wikipedia.org/wiki/The_Legend_of_Zelda_\(video_game\)](https://en.wikipedia.org/wiki/The_Legend_of_Zelda_(video_game))

<https://docs.unity3d.com/6000.3/Documentation/Manual/job-system-parallel-for-jobs.html>

<https://indiegamerchick.com/2025/08/08/zelda1/>

TurboGrafx-16

<https://github.com/TudorBuciuman/Turbografx>

<https://tudorbuciuman.itch.io/> - atât versiunea de Windows, cât și de android

13. Screenshot-uri



Mențiune legală

TurboGrafx: BITROOT este un proiect independent, necomercial, creat exclusiv pentru olimpiada națională de informatică: InfoEducație. Acesta este conceput exclusiv ca un omagiu tehnic și artistic adus moștenirii consolei TurboGrafx-16 și influenței sale asupra dezvoltării jocurilor video.

Acest proiect nu este afiliat, susținut, sponsorizat sau aprobat de NEC Corporation, NEC Home Electronics, Hudson Soft, Konami sau de orice alt deținător al mărcii comerciale TurboGrafx ori al proprietății intelectuale asociate acestora.

„TurboGrafx” și „TurboGrafx-16” sunt mărci comerciale ale deținătorilor respectivi și sunt utilizate **exclusiv** în scopuri **nominative** și **descriptive**, pentru a identifica platforma istorică ce a **inspirat** acest proiect.

Nu se urmărește nicio utilizare comercială. Jocul este non-profit, la o scară foarte mică și folosit exclusiv în acest scop educațional.

2026 Tudor Buciuman. Toate elementele originale – cod, grafică, muzică, dialoguri, design de nivel și orice alt conținut original inclus în TurboGrafx: BITROOT, sunt protejate prin drepturi de autor deținute de autorul respectiv.

Toate drepturile rezervate.